

EC-Fan

2024. 10. 16

1. 서론

□ 연구 배경

- 여러 목적을 동시에 만족해야 하는 다목적 최적화(Multi-Objective Optimization, MOO) 문제를 위하여 알고리즘을 구성함
- 예를 들어, 모터 설계에서는 성능을 최대화 하려는 동시에 리플을 최소화 하여야 하고 동시에 다른 불필요한 토크 변동 현상을 최소화 하여야 하는 복잡한 설계 변수 간의 트레이드 오프 관계가 존재 한다. 이러한 문제에서 단일 해를 찾는 것보다 다양한 트레이드 오프 관계를 만족하는 여러 해를 동시에 찾는 것이 중요하다.
- NSGA-III(Non-dominated Sorting Genetic Algorithm III)은 3개의 상의 목적함수를 다루는 다목적 최적화 문제에서 우수한 성능을 발휘하는 알고리즘이다. reference directions를 사용하여 해집합의 Pareto Front를 효과적으로 탐색할 수 있도록 설계된 알고리즘이다.

□ 연구 목적

- 본 연구의 목적은 NSGA-3 알고리즘을 적용하여 3개의 상충되는 목적함수를 최적화하는 것으로 세 가지 목적함수는 평균 토크(average_T), 토크 리플(Torque_Ripple), 코깅 토크(Cogging_T)로 정의 되며, 이들 간의 최적화는 다양한 설계 변수의 조합을 통해 Pareto 최적 해집합을 도출하는데 중점을 둔다.

2. Basic concept - setup

□ 실험 환경 : 하드웨어/소프트웨어 구성 (사용한 시스템, 라이브러리 등)

- sklearn : 파이썬에서 사용되는 다양한 머신러닝 알고리즘을 제공하는 라이브러리
 - ▶ SVR: 데이터를 회귀 분석하는데 사용되는 비지도 학습 알고리즘
 - ▶ RandomizedSearchCV: 하이퍼파라미터 최적화를 위한 랜덤 탐색을 교차 검증과 함께 수행하는 방법
- pymoo : 다목적 최적화 문제를 해결하기 위한 파이썬 기반 라이브러리
 - ▶ NSGA3: 다목적 최적화 문제에서 사용되는 비지배 정렬 유전 알고리즘
 - ▶ ElementwiseProblem: 문제의 각 요소를 개별적으로 평가하는 방식으로 정의된 최적화 문제 클래스
 - ▶ DefaultMultiObjectiveTermination: 다목적 최적화 문제에 대한 기본 종료조건 설정
 - ▶ FloatRandomSampling: 초기 해 집합을 생성할 때 실수형 변수를 무작위로 샘플링 하는 방법
 - ▶ SimulatedBinaryCrossover(SBX): 유전 알고리즘에서 자주 사용되는 교차 연산자로, 자식 해를 생성할 때 부모 해를 기반으로 새로운 해를 생성
 - ▶ PolynomialMutation: 유전 알고리즘에서 특정 확률로 해의 유전자를 변이 시키는 기법

2. Basic concept - Data preprocessing & Hyperparameter tuning

□ Data Preprocessing

- MinMaxScaler
 - ▶ 정의: 데이터를 [0,1]사이의 값으로 변환하는 방법
 - ▶ 사용 이유: SVR과 같은 모델에서 스케일링된 데이터는 더 좋은 성능을 보임
 - ▶ 공식: $normalized_data = (data - X_{min}) / (X_{max} - X_{min})$
 - ▶ 장점: 모델 훈련 시 속도를 높이고, 하이퍼파라미터 튜닝 효과를 극대화할 수 있음

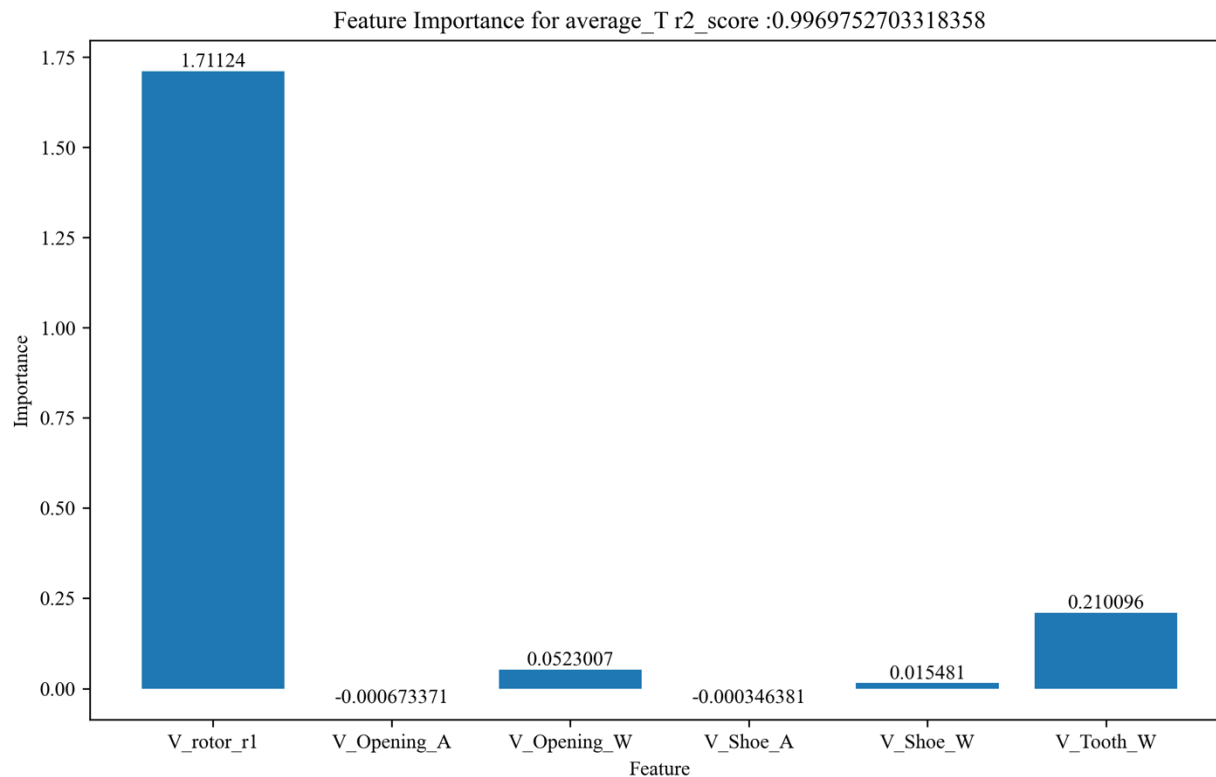
□ Model HyperParameter Tuning

- RandomizedSearchCV
 - ▶ 주어진 하이퍼 파라미터 공간에서 무작위로 조합을 선택하여 최적의 모델을 탐색함
 - ▶ 장점: GridSearchCV보다 더 빠르고 효율적임, 더 많은 하이퍼파라미터 조합을 시도할 수 있어 탐색 범위가 넓어짐
 - ▶ 과정: 임의로 설정된 하이퍼파라미터 조합을 사용하여 여러 번 모델을 학습하고, 교차 검증을 통해 성능을 평가함
 - ▶ 결과: 성능이 가장 좋은 하이퍼파라미터 조합을 반환함.
- SVR의 주요 하이퍼파라미터
 - ▶ C
 - 정규화파라미터로, 큰 값일수록 모델이 훈련 데이터에 더 맞추려고 함
 - 작은 값은 모델을 더 일반화하는 방향으로 유도함
 - ▶ epsilon(ϵ)
 - 오차 허용 범위로, 모델이 무시할 수 있는 오차의 크기를 결정함
 - ▶ kernel
 - 커널 함수로, 데이터의 비선형적 특성을 처리하는 역할을 함.
 - ▶ gamma
 - 'rbf', 'poly', 'sigmoid' 커널에서 사용하는 파라미터로, 데이터 샘플 간의 영향 범위를 결정함
 - 값이 클수록 모델이 복잡해지고, 값이 작을수록 일반화 경향이 커짐

2. Basic concept – Feature Importance

○ Permutation_importance

- ▶ 정의: 모델 훈련 후 변수의 중요도를 평가하는 방법
- ▶ 방법: 각 변수의 값을 무작위로 섞고, 모델 성능에 미치는 영향을 측정
- ▶ 목적: 모델에서 어떤 변수가 가장 중요한지 파악
- ▶ 장점: 모델에 독립적인 방법으로, 모델의 해석 가능성을 높임



2. Basic concept – NSGA-3

□ NSGA-3 알고리즘 개요

- NSGA-3: 다목적 최적화 알고리즘으로, 3개 이상의 목적함수를 최적화할 때 효과적으로 사용됨
- Pareto 최적해: 서로 상충하는 다수의 목표를 최적화하며, NSGA-3는 Pareto Front에서 비 지배 해를 탐색함

□ 주요 구성요소

- `_init_` 함수
 - ▶ `n_var`: 변수 개수 설정
 - ▶ `n_obj`: 목적 함수 개수 설정
 - ▶ `n_constr`: 제약 조건 개수 설정
 - ▶ `xl xu`: lower bound, upper bound 설정
- `_evaluate` 함수
 - ▶ `_evaluate`: NSGA-3의 목적함수와 제약조건을 계산하는 함수
 - ▶ 역할: 개체의 설계 변수(해)에 대해 목적함수 및 제약조건을 평가하여 `out[F]`와 `out[G]`에 저장함
 - ▶ 사용: 이 함수는 개체마다 호출되며, 다목적 최적화 문제에서 중요한 역할을 함
- `Optimize` 함수
 - ▶ `Optimize`: NSGA-3 알고리즘이 실행되는 최적화 루프를 관리하는 함수
 - ▶ 역할: 주어진 문제를 최적화하고, 알고리즘이 각 세대마다 진화하면서 해를 개선해 나가는 과정을 담당
 - ▶ 사용 방법: 목적함수와 제약조건을 반복적으로 평가하며, 해집합을 갱신하고 최적해를 찾음
- `FloatRandomSampling`: 초기 해 생성
 - ▶ `FloatRandomSampling`: NSGA-3에서 초기 해 집합을 생성하는 방법
 - ▶ 역할: 연속형 변수(실수형 변수) 공간에서 무작위로 초기 해를 샘플링하여 최적화 탐색을 시작
 - ▶ 장점: 탐색 공간을 넓게 커버하며, 다양한 해집합으로부터 최적해를 탐색할 수 있도록 도움

2. Basic concept – NSGA-3

○ SimulatedBinaryCrossover(SBX)

- ▶ SimulatedBinaryCrossover: NSGA-3에서 사용되는 교차 연산자로, 부모 해로부터 자식 해를 생성하는 방법
- ▶ 역할: 부모 해들의 유전자 정보를 새로운 자식 해를 생성함으로써 탐색 공간을 확장
- ▶ 파라미터: 'eta' 파라미터는 교차의 범위를 조정하여, 자식 해가 부모 해와 얼마나 가까운지 또는 먼지를 결정
- ▶ 장점: 다양한 해집합을 만들고 더 좋은 해를 탐색할 수 있는 기회를 제공

○ Reference Directions 생성

- ▶ Reference Directions:
 - 다목적 최적화 문제에서 Pareto Front 상의 다양한 해를 균일하게 분산시키기 위해 사용되는 방향 벡터
 - NSGA-3는 다수의 목적함수를 최적화 할 때, 참조 방향을 사용하여 해의 다양성을 보장하고 Pareto Front에서 고르게 분포된 해를 찾음
- ▶ get_reference_directions
 - 역할: 특정 방식으로 참조 방향을 생성하는 함수
 - "das-dennis" 방법: Das-Dennis 방법은 다목적 최적화 문제에서 고차원 해집합의 균등한 분포를 보장하기 위한 기법
 - 'n_obj': 목적함수의 개수
 - n_partitions: 각 차원에서 참조방향을 몇 개의 부분으로 나눌지 결정하는 값으로, 더 많은 분할 수를 사용할수록 해집합이 더 세밀하게 분포됨
- ▶ 참조 방향의 역할
 - 다양성 보장: NSGA-III는 참조 방향을 사용하여 탐색 과정에서 해가 다양한 방향으로 분산될 수 있도록 함
 - Pareto Front 최종 해집합이 다양한 목표를 고르게 최적화할 수 있도록 도움

2. Basic concept – NSGA-3

□ NSGA-3를 사용할 때 회귀 모델을 왜 사용해야 하는가?

○ 목표 함수(Objective Function)의 정의

- ▶ (1) 명확한 수식 기반 목표 함수
 - 수학적으로 잘 정의된 함수가 있다면 그 함수 자체를 바로 NSGA-3에 넣을 수 있음
 - 그러므로 회귀모델은 필요 없음
- ▶ (2) 시뮬레이션/실험 기반 목표 함수
 - 실험이나 시뮬레이션에 의존하는 경우, 목표 함수는 특정 매개변수를 입력했을 때 실험 결과로부터 나오는 값일 수 있음.
 - 이 경우 시뮬레이션 실행이 비용이 크거나 시간이 오래 걸릴 수 있으므로, 이럴 때는 회귀 모델이 유용 할 수 있음
- ▶ (3) 데이터 기반 목표 함수
 - 실험 데이터만 있고, 그로부터 목표 함수를 추론해야 하는 경우도 있는데 이 경우에는 회귀 모델이 필수적임
 - 데이터에서 목표 함수의 패턴을 학습한 후, NSGA-3 모델을 사용하여 최적화 수행

○ 회귀 모델을 사용하는 이유

- ▶ 명시적인 수학적 목표 함수가 없고 데이터 기반으로 목표 함수를 추정해야 하는 상황이기 때문에 사용함
- ▶ 사용할 회귀 모델은 대체 모델(surrogate model)로 작동하는데 실제 실험 대신 데이터를 바탕으로 빠르게 예측을 수행하기 때문에 실험 비용을 줄이고 NSGA-3에서 최적화 과정을 간소화 할 수 있음

2. Basic concept – Surrogate Model

□ 대리모델(Surrogate Model) 개요

- 대리모델은 실제 목표 함수나 시스템을 대신해서 빠르고 저렴하게 근사화 하는 모델임.
- 실제 문제의 계산비용이 매우 크거나 시간 소모가 많을 때, 그 복잡한 시스템을 효율적으로 다루기 위해 대리 모델이 사용됨 특히, 최적화 문제에서 반복적으로 평가해야 할 목표 함수가 매우 복잡하거나 계산 비용이 클 경우, 대리모델은 빠르게 근사화 된 값을 제공하여 최적화 과정의 속도를 크게 높일 수 있음

□ NSGA-3에서 사용되는 대리모델

- NSGA-III 같은 다목적 최적화 알고리즘에서는 대리모델이 복잡한 목표 함수나 제약 조건을 근사화 하기 위해 자주 사용됨.
- NSGA-III는 여러 세대에 걸쳐 많은 평가를 수행하므로, 만약 목표 함수 평가가 비용이 많이 들거나 시간이 오래 걸린다면 최적화가 매우 비효율적일 수 있음. 이를 보완하기 위해 대리모델을 사용하여 목표 함수의 값을 근사화를 함.

□ NSGA-3에서 사용되는 대표적인 대리모델

- 다항 회귀 모델(Polynomial Regression)
- 가우시안 프로세스 회귀(Gaussian Process Regression, GPR)
- 서포트 벡터 회귀(Support Vector Regression, SVR)
- 신경망(Neural Networks)
- 랜덤 포레스트(Random Forest)

2. Basic concept – Surrogate Model

□ 각각의 목적함수에 해당하는 Regressor 모델로 대리모델 생성

- Surrogate Model1 : 'average_T'
- Surrogate Model2 : 'Torque_Ripple'
- Surrogate Model3 : 'Cogging_T'
- Surrogate Model4(constraint) : 'Slot_Area'

□ NSGA-3 대리모델 기반 최적화

- 3개의 목적 함수와 1개의 제약조건을 포함한 최적화 진행

□ 목적함수와 제약조건을의 분배

- 목적함수(F) : 3개의 대리모델을 사용하여 목적함수를 평가
 - ▶ $out[F]$:에 최종 목적함수 값 저장
- 제약조건(G) : 1개의 대리모델을 사용하여 제약조건을 평가
 - ▶ $out[G]$:에 제약조건 위반 정도 값 저장
- 목적함수와 제약조건을의 역할:
 - ▶ 목적함수 : 시스템 성능 최적화
 - ▶ 제약조건 : 설계의 제한 조건을 준수하는지 평가

2. Basic concept – Maximization, Minimization

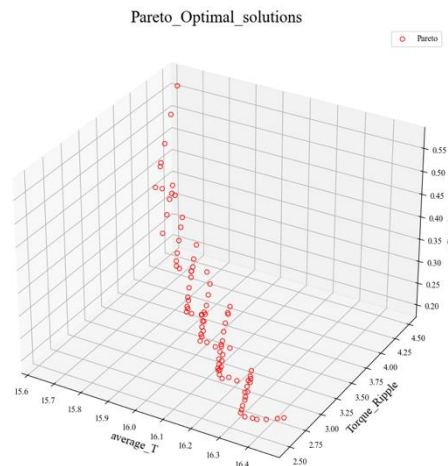
□ 최대화와 최소화 문제의 차이

- 최대화 문제(Maximization Problem)
 - ▶ 목적 함수의 값을 가능한 한 크게 만드는 것이 목표 -> 함수의 최댓값을 찾는 문제
 - 예: 수익을 최대화, 효율을 최대화
- 최소화 문제(Minimization Problem)
 - ▶ 목적 함수의 값을 가능한 한 작게 만드는 것이 목표 -> 함수의 최소값을 찾는 문제
 - 예: 비용을 최소화, 에러를 최소화
- 최대화를 최소화로 변환하기
 - ▶ 최대화 문제를 최소화 문제로 변환하는 방법
 - ▶ 목적함수의 부호를 반대로 만드는 것
 - $\max f_i(x)$: 목적함수가 큰 숫자가 좋은 해
 - $\min -f_i(x)$: 목적함수가 작은 값이 되어야 좋은 해
- NSGA-3
 - ▶ 목표 1: average_T -> Maximization -> 부호 반전(-)
 - ▶ 목표 2 Torque_Ripple -> Minimization
 - ▶ 목표 3 Cogging_T -> Minimization
 - ▶ 목표 4: Slot_Area -> Above 270

2. Basic concept – Pareto Front

□ 파레토 프론트의 개념

- 파레토 최적화(Pareto Optimization): 여러 목표를 동시에 최적화하는 문제에서 하나의 목표를 개선하려면 다른 목표를 희생해야 하는 상황 의미
- 파레토 프론트(Pareto Front): 더 이상 다른 목표를 희생하지 않고는 한 목표를 더 이상 개선할 수 없는 비지배해(Non-dominated Solutions)들의 집합
- 특징 :
 - ▶ 비지배해: 어떤 해도 다른 해보다 모든 면에서 우수하지 않음(두 가지 이상의 목표일때)
 - ▶ 다목적 최적화 문제에서 목표들 간의 트레이드오프를 보여줌
- 파레토 프론트 도출 : NSGA-3를 통해 도출된 해들은 최적화 공간에서 비지배해로 구성된 파레토 프론트를 형성함



3. 실험

□ 실험 설계

○ 실험에 사용된 코드

- ▶ main.py
 - 코드 전반적인 흐름 관여
 - Surrogate modeling과 NSGA-3 Algorithm 전달하는 역할 수행
- ▶ Model.py
 - 제한 제약 조건 설정
- ▶ leaner.py
 - MOO 의 Base가 되는 surrogate SVR 모델 학습
- ▶ Optimizer.py
 - NSGA-3 3개의 목적함수와 1개의 제약 조건으로 최적화 수행

○ 사용된 알고리즘

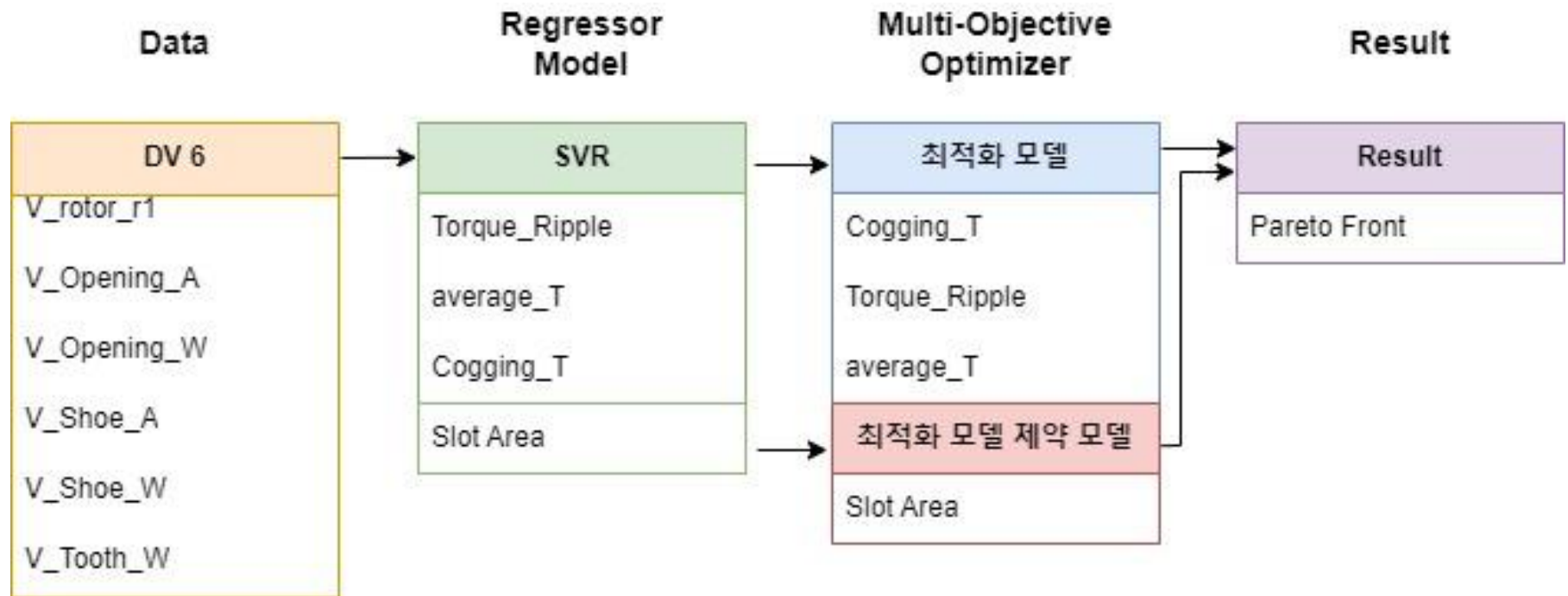
- ▶ 서포트 벡터 회귀(Support Vector Regressor, SVR)
 - Surrogate Model 4개 (obj 3개, cons 1개)
- ▶ NSGA-3
 - Multi-objective optimization – Class Moo

○ 데이터 구성

- ▶ 20240819_2th_ECmotor_SuperCom_loadcase_total.csv
- ▶ EC Fan의 데이터는 총 10개의 데이터로 이루어져 있다. [6720 rows x 10 columns]

열 번호	이름	설명
1	V_rotor_r1	Design Variable(설계 변수) 1
2	V_Opening_A	Design Variable(설계 변수) 2
3	V_Opening_W	Design Variable(설계 변수) 3
4	V_Shoe_A	Design Variable(설계 변수) 4
5	V_Shoe_W	Design Variable(설계 변수) 5
6	V_Tooth_W	Design Variable(설계 변수) 6
7	average_T	최적화 대상 목적함수 1 (평균토크) Maximization
8	Torque_Ripple	최적화 대상 목적함수 2 (토크 리플) Minimization
9	Cogging_T	최적화 대상 목적함수 3 (코깅 토크) Minimization
10	Slot_Area	제약 조건 Above 270

□ Flowchart



4. 실험 과정

본 실험에서는 데이터셋에서 6개의 Design Variable은 그대로 두고, 3개의 목적함수('average_T', 'Torque_Rippe', 'Cogging_T')에 대해서는 MinMax Scaling을 적용하였다.

또한, 목적함수 'average_T'는 최대화를 목표로 하였기 때문에 y값을 음수로 변환하여 최적화를 진행하였다.

데이터는 `train_test_split`을 사용하여 훈련 데이터와 테스트 데이터로 분리하였으며, SVR 모델의 하이퍼파라미터 튜닝은 `RandomizedSearchCV`를 통해 수행 되었다.

사용한 parameter search Grid는 다음과 같다.

- **C**: [0.01, 0.1, 1, 10, 100]
- **kernel**: ['poly', 'rbf']
- **gamma**: ['scale', 'auto', 0.001, 0.01, 0.1, 1]
- **epsilon**: [0.001, 0.01, 0.1, 0.5, 1]

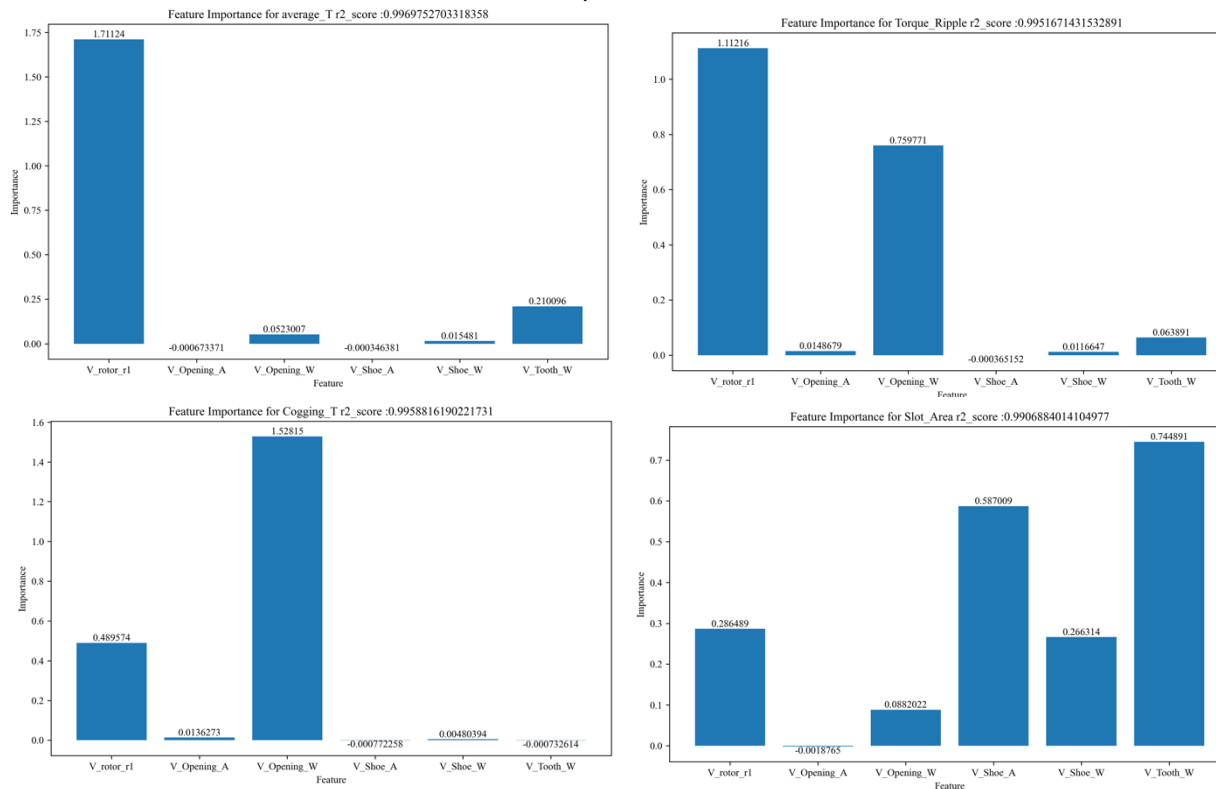
`RandomizedSearchCV`에서는 SVR 모델을 사용하였고, `param_dist` 를 위와 같이 설정한 후, `n_iter=5`, `scoring='neg_mean_squared_error'`, `cv=5`로 하이퍼파라미터 탐색을 진행하였다.

최종적으로 최적의 SVR 하이퍼파라미터는 `SVR(C=10, kernel='rbf', epsilon=0.01, gamma=1)`로 도출되었으며, 4개의 목적함수 모두에서 최적 파라미터를 얻었다.

최적 파라미터를 바탕으로 모델을 학습한 후, 테스트 데이터로 사용하여 예측한 결과, 각 목적함수에 대한 R^2 score 값은 다음과 같이 나타났다.

- **Average_T**: 0.9970
- **Torque_Ripple**: 0.9952
- **Cogging_T**: 0.9958
- **Slot_Area**: 0.9907

각 모델에서 변수의 특징 중요도를 평가하기 위해 Permutation_importance를 적용하였으며, 그 결과는 아래와 나타났다.



□ 모델평가 : Permutation Importance 분석

변수	average_T	Torque_Ripple	Cogging_T	Slot_Area
V_rotor_r1	1.7112	1.1122	0.4896	0.2865
V_Opening_A	-0.0007	0.0149	0.0136	-0.0019
V_Opening_W	0.0523	0.7598	1.5282	0.0882
V_Shoe_A	-0.0003	-0.0004	-0.0008	0.587
V_Shoe_W	0.0155	0.0012	0.0048	0.2663
V_Tooth_W	0.2101	0.0639	-0.0007	0.7449

- 분석 결과, 'average_T'에서는 V_rotor_r1이 'Torque_Ripple'에서는 V_rotor_r1과 V_Opening_W, 'Cogging_T'에서는 V_Opening_W, 'Slot Area'에서는 V_Tooth_W/V_Shoe_A이 목적함수에서 높은 중요도를 보이며, 모델의 성능에 크게 기여한것으로 확인 되었다. V_Opening_A는 상대적으로 낮은 중요도를 나타내, 해당 변수가 모델에서 차지하는 비중이 적다는 것을 알 수 있다.

□ 대리 모델 구성

- 본 연구에선는 대리 모델을 통해 실험 데이터를 학습하고, NSGA-3 알고리즘의 최적화 효율을 높였다. 대리모델은 고비용 시뮬레이션 대신 예측 모델을 사용하여, 최적화를 더 빠르게 진행 할 수 있도록 설계 하였다. 목적함수는 'average_T', 'Torque_Rippe', 'Cogging_T'으로 두고 제약 조건은 'Slot_Area'로 정의 하였으며, 이를 통해 다목적 최적화를 효과적으로 수행하였다.

□ NSGA-3 알고리즘의 구성 및 최적화 과정

항목	내용
목적함수	3개의 목적 함수 사용 'average_T', 'Torque_Rippe', 'Cogging_T'
제약 조건	1개의 제약 조건 사용 'Slot_Area' -> Above 270
평가 함수 (_evaluate)	목적함수와 제약조건의 적합도(fitness)와 위반 여부 평가
최적화(optimize)	각 세대에서의 최적의 해를 반복적으로 찾으며 최적화 진행
종료조건(termination)	'xtol', 'ftol', 'cvtol' 을 설정하여 최적화가 설정된 기준에 도달 시 종료
참조 방향(ref_dirs)	참조 방향을 설정하여 다양한 Pareto 해를 탐색
샘플링(sampling)	초기 해집단을 생성하고, 그 후 교차(crossover) 및 변이(mutation) 연산 수행
중복해 제거 (eliminate_duplicates)	중복된 해를 제거하여 탐색 공간의 다양성을 유지함
최종 결과(res)	최적화 결과로부터 최종 해 반환 및 Pareto Front 도출

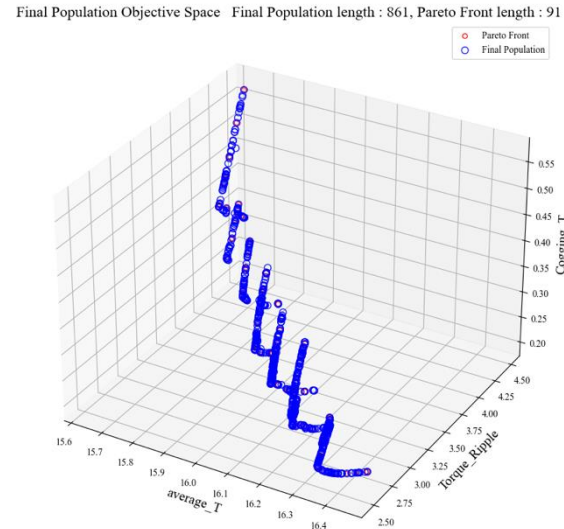
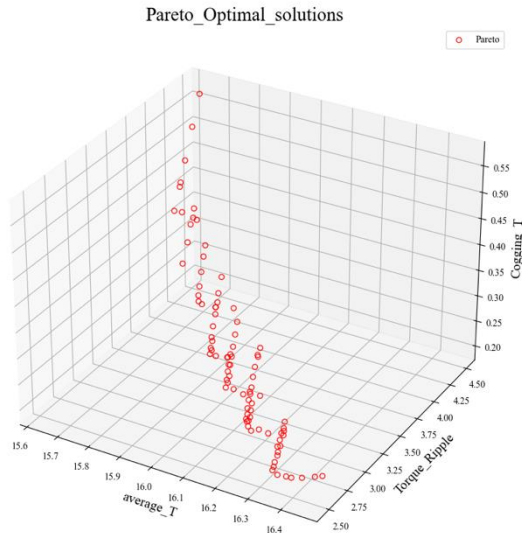
□ 하이퍼파라미터 튜닝 및 최적화 설정

- NSGA-3의 성능을 최적화하기 위해, 각 단계에서 사용한 설정은 아래와 같다.

설정 항목	설명
초기 설정	n_var=6, n_obj=3, n_constr=1, 'xl, xu'=model의 조건 사용
_evaluate	대리모델(Surrogate Model) 결과를 사용하여 SVR 모델을 적용하였으며, out['F']에 3개의 목적함수를 넣고, out['G']에 1개의 제약조건을 포함하였다.
참조 방향(ref_dirs)	Pareto 최적해 탐색을 위한 참조 방향 설정, "das-dennis", n_partitions는 40으로 pop_size는 861로 설정
교차율(crossover rate)	SimulatedBinary Crossover를 사용하였고 0.9의 교차율과 eta를 15로 설정
변이율(mutation rate)	PolynomialMutation를 사용함
종료 조건(termination)	xtol=1e-8, ftol=0.0025, cvtol=1e-6의 종료 조건을 설정하여 3000 세대까지 최적화를 진행함.

5. 결과 분석

□ Pareto Front 시각화 및 결과 분석



- Pareto Front와 최종 집단의 관계
 - ▶ 최종 집단은 파란색으로 표시 되었으며, 빨간색으로 표시된 해는 Pareto Front에 속한 최적 해들이다.
- 경계선: 파레토 프론트가 비지배해로서 경계를 형성했다.
- 비지배해가 고르게 분포: 다양한 트레이드오프를 찾아냈음을 확인했고 다양한 목표 조합에 대해 최적화된 해를 찾았다.
- 특정한 패턴: 파레토 프론트가 계단식(step-like pattern)으로 나타나 각 목표 간의 트레이드 오프를 잘 반영 했다.
- 파레토 프론트가 명확한 경계를 형성하고, 다양한 트레이드오프가 잘 반영된 해들을 포함하고 있기 때문에 최적화과 효과적으로 수행되었고, 각 목표 간의 최적 타협점들이 잘 도출되었다.

6. 결론

이번 실험에서는 SVR(Support Vector Regressor)을 대리모델로 사용하여 NSGA-3 알고리즘을 통해 3개의 목적함수를 최적화 했다. 처음에는 XGBoostRegressor와 같은 트리 기반 회귀 모델을 사용했지만, 트리 기반 모델이 좁은(제한) 범위에서 최적해를 찾는 경향이 있어, 보다 일반적이고 다양한 해를 찾기 위해 SVR 모델을 사용하게 되었다. SVR 모델은 특정한 제한 조건 없이 넓은 해 탐색이 가능하다는 장점이 있어 최적화 과정에서 더 적합한 대안이어서 선택되었다.

최적화 결과, 파레토 프론트는 명확한 경계를 형성하며 각 목표 간의 다양한 트레이드오프를 잘 반영한 해를 얻을 수 있었다. 6개의 설계 변수와 1개의 제약 조건을 적용한 결과 Slot_Area라는 제약조건(270 이상)을 만족하는 861개중 91개의 최적해를 찾을 수 있었다.

본 연구는 3개의 목적 함수를 동시에 최적화 하는 문제에서 비지배해를 성공적으로 도출함으로써 다목적 최적화 문제의 해결에 있어서 성과를 얻을 수 있었다.